
現場報告書AI処理・ナレッジ検索システム

Report Insight 運用ハンドブック

壊れたときに、何を見て・どう直すか。監視／障害対応／デプロイの実務指針

対象：開発・インフラ運用担当 / SRE

前提：AWS(ECS/RDS/SQS/S3) + Terraform + GitHub Actions

運用者向け

〇〇ビルメンテナンス株式会社

運用者が押さえるべきは「非同期パイプライン」と「LLM依存」

本システムは①S3→SQS→Workerの非同期取込と②LLM(Claude API)への外部依存を持つ。一般的なWeb運用に加え、この2点の“詰まり”と“外部障害”の勘所を先に共有する。

非同期の詰まり

- SQSにメッセージが滞留
- DLQ（失敗キュー）に落ちる
- → Worker/DB/LLMのどこが原因かを切り分ける

外部LLM依存

- Claude APIの5xx / レート超過
- レイテンシ悪化・コスト急増
- → 503+Retry-Afterで縮退、監視でトークン量を追う

品質の劣化

- 分類精度・構造化失敗率の低下
- プロンプト変更が回帰を招く
- → CIの回帰評価ゲートでマージ前に検知

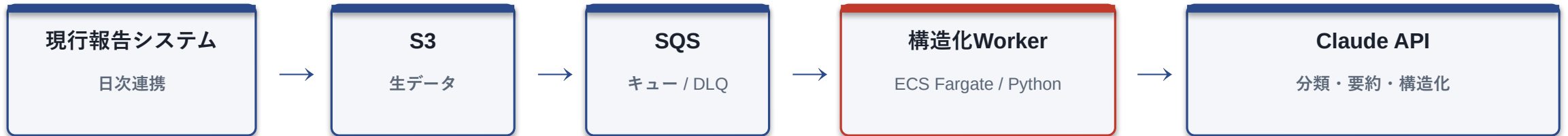
01

システム全体像と処理フロー

どのコンポーネントが、どの順で、何をするか。障害切り分けの地図として。

全体アーキテクチャ — 取込系(Worker)とAPI系を分離

■ 取込・構造化（非同期）



■ 検索・報告書生成（同期API）



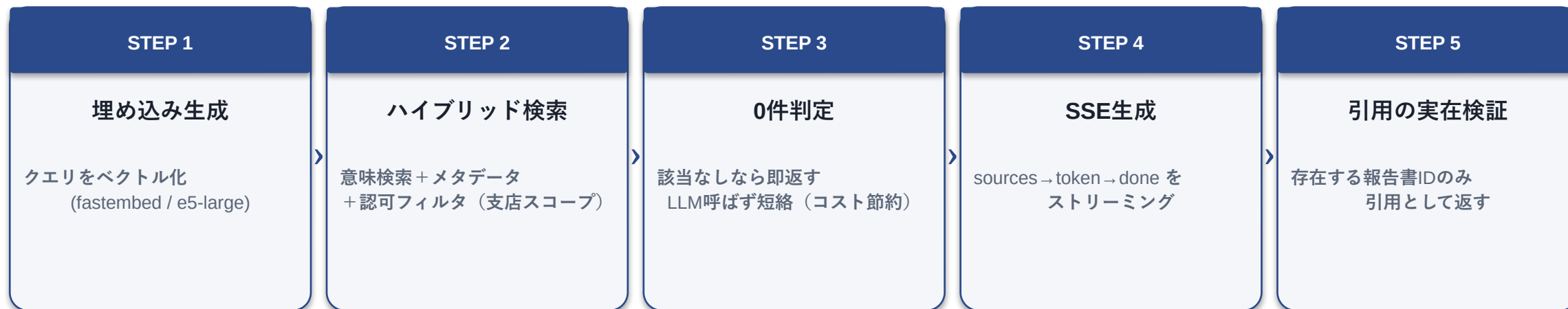
切り分けの起点：赤=Worker / 藍=API / 緑=DB。
症状がどのラインで起きているかをまず特定する。

Workerが緊急度「高」を検出 → Slack/メールへ即時Webhook通知

取込パイプラインの内部処理と“止まりどころ”

段階	処理	失敗時の挙動 / 監視ポイント
1. 受信	SQSからメッセージ取得（冪等）	重複はUPSERTで吸収。滞留=Worker能力/詰まり
2. PIIマスキング	個人名・電話番号を除去してからLLM送信	マスキング漏れは重大。事前検証必須
3. 分類(structured)	category/urgency/action_required を構造化出力	パース失敗=構造化失敗率に計上
4. 確信度判定	閾値未満は needs_review に倒す	しきい値はチューニング対象
5. 緊急通知	urgency=high は Webhook 即通知	通知失敗はリトライ、監視で検知
6. 保存	冪等UPSERTでDBへ	失敗は再試行→最終的にDLQ

RAG検索フロー — ハルシネーションとコストの防波堤



運用上の意味：0件短絡は“無駄なLLM課金”を止める仕組み。引用の实在検証はAPI層で行うため、検証をすり抜けた引用が出た場合はAPI側のバグを疑う。

性能目標：検索応答3秒以内（生成部はストリーミング表示で体感を担保）。

02

監視・障害対応・デプロイ

日々どこを見るか。アラートが鳴ったら何をするか。安全に出す・戻す。

監視：毎朝この4指標を見る（CloudWatch）

指標	意味	正常の目安	外れたら疑うこと
構造化失敗率	LLM構造化のパス/処理失敗の割合	低位で安定	プロンプト変更 / LLM側障害 / 入力異常
APIエラー率	FastAPIの5xx割合	1%未満	DB接続 / LLM 503 / デプロイ直後の退行
トークン消費	Claude APIの入出力トークン	日次予算内	検索急増 / 0件短絡の不発 / プロンプト肥大
SQS滞留 / DLQ	未処理件数と失敗キュー	滞留ゼロ・DLQ空	Worker停止 / 例外多発 / スロットリング

これらは非機能要件（コスト月10万円 / 分類精度90% / 応答3秒）の健康診断。閾値超過はSNS経由でアラート通知。

アラートが鳴ったら — 症状別の初動

アラート	まず見る	初動アクション
SQS滞留 / DLQ増	Workerタスク数・Workerログ・例外	Worker再起動→原因例外を特定→DLQを再投入
APIエラー率↑	対象エンドポイント・直近デプロイ	退行ならロールバック / DB・LLM起因なら縮退確認
LLM 503 多発	Claude APIステータス・Retry-After	縮退(503応答)継続を確認・復旧待ち・必要ならモデル切替
トークン急増	検索リクエスト数・0件短絡率	異常クエリ源を特定→レート制限(10req/min)確認
構造化失敗率↑	直近のプロンプト/モデル変更	変更をロールバック→回帰評価で再確認

原則：外部(LLM)障害は“縮退して待つ”、内部退行は“戻す（ロールバック）”。切り分けの起点は「直近にデプロイしたか？」。

デプロイ／ロールバック — 安全に出す・すぐ戻す

パイプライン (GitHub Actions)

- test → LLM回帰評価 → build → deploy
- 回帰評価が閾値割れならマージ/デプロイを止める
- 認証はOIDC (静的キーを置かない)

ブランチ保護と環境

- main保護 + レビュー必須
- dev環境で先行検証してからprod
- IaCはterraform plan差分をレビュー

ロールバック

- ECSは前タスク定義に戻す
- DBはAlembicのdowngradeを事前確認
- プロンプト/モデルはgit revertで即戻す

リリース判断の勘所

- LLM関連変更は回帰評価green必須
- デプロイ直後はAPIエラー率を注視
- 疑わしきはロールバック優先

セキュリティ — 社外秘 × LLM の勘所

データ保護

- LLM送信前にPIIマスキング
(個人名・電話番号)
- IP制限 + SSO想定
- 認可フィルタは検索・一覧の
DBクエリ段で強制

DevSecOpsゲート

- SAST / SCA / IaCスキャン
- コンテナイメージスキャン
- 脆弱性トリアージSLAで対応期限管理
- クレデンシャルは静的キーゼロ(OIDC)

LLM固有リスク

- プロンプトインジェクション対策
- 出力を鵜呑みにしない
(引用の实在検証)
- 監査ログ (承認操作等) を記録
- OWASP LLM Top10を設計で参照

コスト管理 ― 月10万円を超えさせない仕組み

コストを抑える設計

- ・ タスク別にモデルを使い分け
- ・ 検索0件は短絡してLLMを呼ばない
- ・ プロンプト肥大を避ける

監視で守る

- ・ トークン消費をCloudWatchで実測
- ・ 日次コストアラート
- ・ 超過源をクエリ単位で特定

コスト急増を見たときの分解

① 件数が増えたのか（検索/取込のリクエスト数） ② 1件が重くなったのか（プロンプト・入力トークン肥大） ③ 0件短絡が効いていないのか（無駄なLLM呼び出し）

→ この3軸で切り分ければ、原因は必ずどれかに落ちる。

定常運用チェックリスト

この頻度で回せば、障害は“予兆”のうちに拾える。

毎日

- ☐ 4指標ダッシュボード確認
(失敗率/APIエラー/トークン/DLQ)
- ☐ DLQ件数がゼロか
- ☐ コストアラートの有無

毎週

- ☐ 未分類キュー滞留の傾向
- ☐ 分類精度の推移レビュー
- ☐ 脆弱性スキャン結果のトリアージ

リリース時

- ☐ 回帰評価がgreenか
- ☐ デプロイ直後のエラー率監視
- ☐ ロールバック手順の再確認

次アクション：Runbook（詳細手順書）とオンコール体制の整備、AWS dev環境での障害訓練（P2）。